

Programming

COMP1850

Module handbook
2026/27

Module Leader:

Dr Mark Walkley (module leader)

Module Staff:

Mx Amy Brereton (Lecturer)
Dr Hui Lau (Lecturer)
Dr Nick Efford (Lecturer)
Dr Thomas Richardson (Lab support)
Dr Qian Xie (Lab support)
Dr Ban Adil Naji Al-Jassani (Lab support)
Mr David Ofilli (Lab support)



UNIVERSITY OF LEEDS

Preface

Programming (COMP1850) was first developed in 2024 as part of the University of Leeds Curriculum Redefined project. This module is a core part of the curriculum for the following degree programmes;

- BSc (Hons) Computer Science,
- BSc (Hons) Computer Science (Industrial)
- BSc (Hons) Computer Science (International)
- BSc (Hons) Computer Science (Artificial Intelligence)
- MEng, BSc (Hons) Computer Science
- MEng, BSc (Hons) Computer Science (Industrial)
- MEng, BSc (Hons) Computer Science (International)
- MEng, BSc (Hons) Computer Science (Artificial Intelligence)
- MEng, BSc (Hons) Computer Science (Artificial Intelligence) (Industrial)
- MEng, BSc (Hons) Computer Science (Artificial Intelligence) (International)
- MEng, BSc (Hons) Computer Science (High-performance graphics & Games engineering)
- MEng, BSc (Hons) Computer Science (High-performance graphics & Games engineering) (Industrial)
- MEng, BSc (Hons) Computer Science (High-performance graphics & Games engineering) (International)

The module is designed in consideration of the ACM/IEEE Computer Science 2023 curriculum and mapped against the requirements for Partial Chartered IT professional and Chartered IT professional (AHEPv4).

The module is the responsibility of the School of Computer Science, Faculty of Engineering & Physical Sciences, University of Leeds.

Contents

Preface	
1 Module overview	1
Aims	1
Why is this module important?	1
Learning outcomes	2
2 Logistics	4
Teaching approach	4
Lab and tutorial classes	4
Skills development	4
Module Schedule	5
Access to learning resources	7
Support opportunities	7
Module survey & Evaluation	7
Student voice	7
3 Attendance & Engagement expectations	8
Attendance expectations	8
Participation requirements	8
Conduct in lectures, tutorials and laboratories	8
Monitoring engagement	10
Consequences of poor attendance	10
4 Indicative Course Content	11
Unit 1: Programming principles	11
Unit 2: Python programming	11
Unit 3: C programming	12
Unit 4: Databases and SQL programming	14
5 Assessment & Feedback	15
Assessments structure	15
Feedback approach	16
Understanding your grade	16
Assessment timeline & Late submissions	17
Resit opportunity	17
6 Academic Integrity	19
Academic integrity statement	19
Use of assistive technologies	19
7 Reading & Additional resources	20
Reading list	20
Additional resources	20
8 Accessibility	21
Accessibility statement	21
Requesting reasonable adjustments to learning resources	21
Requesting reasonable adjustments in assessments	21

1 Module overview

COMP1850 - Programming introduces the fundamental concepts, techniques and practices required to design, develop, test and maintain software. As a core component of the Computer Science curriculum, the module develops students' ability to approach programming systematically, combining practical programming experience with an understanding of algorithms, data structures, data modelling and software design.

The module covers essential programming concepts, including language syntax and semantics, control structures, design, testing and debugging, as well as the characteristics of compiled and interpreted languages. Students also develop an understanding of how data is represented and manipulated through the study of data types and data structures. An introduction to database systems provides a further foundation in data modelling, database design and implementation.

Through a combination of theoretical teaching, practical exercises and structured problem-solving activities, students develop algorithmic thinking and the ability to design and implement software solutions to complex problems. Particular emphasis is placed on selecting and using appropriate tools and techniques to design, test, analyse and evaluate software, while recognising issues relating to security, reliability, performance and risk.

The module also considers the wider responsibilities associated with software development. Students are introduced to ethical considerations in the design and implementation of software and are expected to make reasoned decisions informed by relevant ethical frameworks and professional codes of conduct. Communication, teamwork, time management, critical thinking, reflection and active learning are developed alongside technical programming skills.

COMP1850 provides an important foundation for subsequent study in computer science and software engineering. By the end of the module, students should be able to apply programming and engineering principles to the development of software artefacts, evaluate their solutions, communicate technical ideas to both technical and non-technical audiences, and identify areas for further development of their knowledge and skills.

Aims

The aim of COMP1850 Programming is to provide you with a strong foundation in programming and software development, developing your ability to analyse computational problems, design and implement effective software solutions, and systematically test and evaluate your work. The module introduces key concepts including programming languages, algorithms, data structures and data modelling, while developing the problem-solving, critical thinking and practical skills needed for further study in Computer Science and for professional software development.

Why is this module important?

Programming is a foundational activity within computer science. It provides a practical means through which you can explore and apply many of the discipline's central ideas, while also developing the computational thinking and problem-solving skills needed throughout your degree. COMP1850 therefore provides an important foundation for your subsequent study by helping you move from understanding computational concepts to applying them to the development of working software.

The importance of the module extends beyond learning how to write programs. Effective programming requires you to analyse problems, identify relevant information, decompose complex tasks, design solutions, make decisions about alternative approaches, and evaluate whether a solution meets its intended requirements. These are fundamental ways of thinking within computer science and are applicable across areas such as algorithms, artificial intelligence, data science, software engineering, computer systems and cybersecurity.

The module also contributes to the development of you as a well-rounded computer scientist by introducing you to software development as a broader discipline. You will consider how software is designed, tested, evaluated and maintained, as well as how factors such as performance, reliability, security, risk and ethical considerations can influence technical decisions. This encourages you to recognise that successful software development is not simply about producing code that works, but about understanding the context in which software is developed and the consequences of the decisions made during its development.

COMP1850 also provides opportunities to develop skills that are important beyond individual programming tasks. Through practical problem-solving and software development activities, you will develop persistence, attention to detail and the ability to learn from errors. Debugging, testing and evaluating your work require you to adopt a systematic and reflective approach, encouraging you to question assumptions and use evidence to improve your solutions. These practices contribute to the development of critical thinking and an ability to deal constructively with unfamiliar and complex problems.

The module supports your development as an independent learner. Programming is a domain in which there is rarely a single prescribed route to a solution, and developing effective solutions often requires you to investigate documentation, experiment with approaches, interpret feedback and identify gaps in your own understanding. By engaging with these activities, you will begin to develop the self-directed learning habits required to continue developing your technical knowledge throughout your degree and beyond graduation.

Programming also provides a context in which you can develop communication and collaboration skills. Software is rarely developed in isolation, and computer scientists need to communicate technical ideas, justify design decisions and work effectively with others. The module therefore contributes to your ability to explain computational ideas clearly, engage with feedback and collaborate when addressing technical problems.

These capabilities are directly relevant to graduate development. Employers across the computing sector require graduates who can combine technical knowledge with analytical thinking, problem-solving, communication, collaboration and the ability to adapt to new technologies and working practices. COMP1850 begins this development by giving you experience of applying knowledge to authentic computational problems while encouraging you to reflect on the quality, implications and limitations of your solutions.

The module therefore contributes to the development of a broad set of graduate capabilities. You will develop technical competence alongside computational thinking, analytical reasoning, problem-solving, communication, collaboration, independent learning and professional awareness. Together, these capabilities provide a foundation for further study within computer science and support your progression towards becoming a capable, adaptable and reflective computer science graduate.

Learning outcomes

On successful completion of this module a student will have demonstrated the ability to:

1. apply subject specific knowledge and engineering design principles to design and implement software artefacts which satisfy complex real-world requirements (C1, M1, C2, M2, C5, M5, C6, M6)
2. select and interpret sources of information to solve complex real-world problems. (C4, M4)
3. use appropriately selected tools to design, test, analyse and evaluate computer systems and identify limitations (C12, M12, C13, M13)
4. identify and analyse ethical concerns when designing and implementing software and make reasoned decisions informed by ethical frameworks and codes of conduct. (C8, M8)
5. identify and interpret risk concerning security, reliability and performance and plan a mitigation approach (C10, M10)
6. effectively communicate complex topics related to programming and software engineering to technical and non-technical audiences. (C17, M17)
7. reflect on their level of mastery of subject knowledge and skills and plan for personal development. (C18, M18)

2 Logistics

Teaching approach

The module has been designed to be interactive and engaging. All instructor led activities are on campus and attendance is expected. During instructor led sessions you will be engaged in learning through direct delivery from a member of teaching staff in a lecture theatre, computer lab or tutorial room. During these sessions you may also be asked to complete exercises, work in small groups, discuss a topic, contribute to a discussion, engage in online polling and otherwise make contributions to the session. If you have reasonable adjustments as part of a disability support summary then please make yourself known to the member of staff delivering the session. The active approach to learning employed by this module means you will get the most out of the instructor led sessions by engaging in all activities, if you are unsure about the activity then please ask the instructor.

Each week you will be given an activity sheet which contains activities for you to complete during your independent study time. Some of the exercises on the activity sheet form part of your portfolio of evidence and should be submitted via Minerva. During lab classes and tutorials the activity sheet will be discussed. Sample solutions will be provided for all formative exercises on the activity sheets.

Lab and tutorial classes

Lab and tutorial classes provide an opportunity to apply the concepts and techniques introduced during the instructor led sessions. These sessions are designed to be practical and participatory, giving you time to work through problems, complete exercises and develop your understanding with support from the teaching team.

During lab classes you can expect to work through practical activities, exercises and examples, often using relevant software, tools or other resources. You may be asked to work individually or collaboratively with other students. Teaching staff will be available to provide guidance, answer questions, clarify concepts and help you identify and address areas where you may need further development. You should come prepared to participate actively and to discuss your approach to the activities with the teaching team and your peers.

Tutorial classes provide an opportunity to explore topics in greater depth and to develop your ability to apply your knowledge to different problems and scenarios. You may be asked to discuss questions, explain your reasoning, work through problems, review your answers or contribute to group activities. Tutorials are also an opportunity to receive feedback on your understanding and to identify areas for further independent study.

You should use both lab and tutorial classes as opportunities to ask questions and test your understanding. You are not expected to know how to complete every activity before attending; making mistakes and discussing different approaches are important parts of the learning process. To get the most from these sessions, you should review the relevant material beforehand, attempt the activities on the weekly activity sheet and actively engage with the tasks during the session.

Skills development

As well as developing your subject knowledge, this module will support the development of a range of academic, practical and transferable skills. The activities undertaken during instructor led sessions, labs, tutorials and independent study are designed to give you opportunities to practise these skills and apply them in different contexts.

You will develop your ability to analyse problems, select appropriate approaches and apply your knowledge to reach and justify solutions. You will also develop your ability to work independently, manage your time and take responsibility for your own learning through the completion of weekly activities and portfolio evidence.

Where activities involve working with other students, you will have opportunities to develop communication, collaboration and teamwork skills. You may also be required to explain your ideas, discuss alternative approaches and provide or respond to feedback. These activities are intended to help you become more confident in communicating technical or subject-specific information to different audiences.

The module will also provide opportunities to develop practical and digital skills relevant to the subject. Through lab activities and other practical exercises, you will gain experience of using appropriate tools, techniques and resources, while developing the ability to evaluate your own work and identify areas for improvement.

You should view feedback from teaching staff, sample solutions and your own reflection on completed activities as important parts of your skills development. Regularly reviewing your work and acting on feedback will help you to develop greater confidence, independence and proficiency throughout the module.

Module Schedule

This is indicative of content and organisation but we reserve the right to reorder material if necessary and/or appropriate.

Week	Topic	Instructor
Semester 1		
1	Introduction to the module and Python	Dr Mark Walkley Dr Hui Lau Dr Nick Efford
2	Data	Dr Mark Walkley Dr Hui Lau Dr Nick Efford
3	Flow control	Dr Mark Walkley Dr Hui Lau Dr Nick Efford
4	Functions	Dr Mark Walkley Dr Hui Lau Dr Nick Efford
5	Debugging and testing	Dr Mark Walkley Dr Hui Lau Dr Nick Efford

6	Object-oriented programming	Dr Mark Walkley Dr Hui Lau Dr Nick Efford
7	Object-oriented programming	Dr Mark Walkley Dr Hui Lau Dr Nick Efford
8	Libraries	Dr Mark Walkley Dr Hui Lau Dr Nick Efford
9	APIs	Dr Mark Walkley Dr Hui Lau Dr Nick Efford
10	Database systems	Dr Mark Walkley Dr Hui Lau Dr Nick Efford
11	Relational databases and SQL	Dr Mark Walkley Dr Hui Lau Dr Nick Efford
Semester 2		
1	SQL programming	Mx Amy Brereton Dr Hui Lau Dr Nick Efford
2	Introduction to C	Mx Amy Brereton Dr Hui Lau Dr Nick Efford
3	Data representation	Mx Amy Brereton Dr Hui Lau Dr Nick Efford
4	Memory and pointers	Mx Amy Brereton Dr Hui Lau Dr Nick Efford
5	Flow control	Mx Amy Brereton Dr Hui Lau Dr Nick Efford
6	Functions	Mx Amy Brereton Dr Hui Lau Dr Nick Efford
7	Debugging and testing	Mx Amy Brereton Dr Hui Lau Dr Nick Efford
8	Structures and modular programming	Mx Amy Brereton Dr Hui Lau Dr Nick Efford
9	Data structures	Mx Amy Brereton Dr Hui Lau Dr Nick Efford
10	Profiling and code optimisation	Mx Amy Brereton Dr Hui Lau Dr Nick Efford
11	Course review	Mx Amy Brereton Dr Hui Lau

Access to learning resources

All learning resources will be made available on the Minerva virtual learning environment prior to the session. Lecture recordings, where they exist, will usually be available via the Minerva virtual learning environment platform within 72 hours of end of the activity. Some of the discursive aspects of the sessions may not be recorded and lecture capture is not a replacement for attendance.

All resources are subject to the accessibility statement (See Accessibility statement).

Support opportunities

All of the teaching activities for this module can be found in your personalised timetable on the LeedsUni app. The module timetable that displays all activities for the module is available at the following link.

Module Timetable View

In addition to timetabled activities you can request additional support from the module teaching staff. You can book one-to-one and small group support sessions using the online booking system.

Module leaders and module staff also host a weekly office hour (possibly on request) where you can seek support directly from the module teaching team.

For support with any mathematical topic in this module you can attend Maths lunch hours which are hosted on **Monday** and **Wednesday** at 1300 in Bragg GR.18. Alternatively, you can request help from Skills@library.

Module survey & Evaluation

As part of the review and development of this module we will ask you to provide feedback on the module. We will do this at least twice during the module; once in December and once in May. Your feedback will be used to improve your experience on the module and will feed into the annual review process of modules in the School of Computer Science.

A summary of the module survey outcomes as well as an action plan will be published for you to review after the module evaluations have been processed. If you have queries about the Module Survey & Evaluations then please speak to the module leader.

Student voice

The student experience and the student voice is important in the review and development of the delivery and content of the module. If you have any comments, areas of improvement or commendations then please speak to your course representative who will represent you at the Student:Staff Partnership forum.

3 Attendance & Engagement expectations

Attendance expectations

You are expected to attend all timetabled sessions associated with your modules and programme. This includes lectures, tutorials, practical sessions, laboratories, workshops, seminars, group activities, and any other scheduled teaching or learning activities.

Regular attendance is an important part of your successful study. Attending timetabled sessions gives you opportunities to develop your subject knowledge and practical skills, ask questions, receive feedback, and work through material with teaching staff and other students. For a computer science degree, attendance is particularly important for practical and collaborative activities, where you develop your learning through hands-on work, problem solving, and interaction with others.

Attendance also contributes to your wider student experience. Being present on campus gives you opportunities to build relationships with other students, develop a sense of belonging to your programme and the University community, and participate in academic and social activities. Regular attendance can also help staff identify when you may be experiencing difficulties and provide appropriate support at an early stage. Attendance is therefore not simply an academic expectation; it is also an important part of your wellbeing and of making the most of your university experience.

If you are unable to attend a timetabled session, for example because of illness or other circumstances affecting your ability to study, you should follow the University's procedures for reporting an absence and, where appropriate, seek advice and support.

Participation requirements

You are expected to actively participate in all scheduled learning activities and to engage consistently with your studies more generally. Attendance is an important starting point, but being present is not in itself sufficient: you should take an active role in your learning.

Active participation may include preparing for sessions, completing assigned activities, contributing to discussions, attempting practical exercises, working constructively with your peers, asking questions, seeking and responding to feedback, and completing independent study and assessment activities by the required deadlines.

You are also expected to engage with the wider University experience and to take reasonable responsibility for your own academic progress and development. This includes making effective use of the learning resources, support services, academic guidance, and opportunities available to you.

Your university experience is an active process, and you generally get out of your degree what you put into it. Consistent engagement will help you develop not only your technical and academic knowledge, but also the communication, collaboration, problem-solving, and professional skills that are important for a career in computer science.

Conduct in lectures, tutorials and laboratories

You are expected to conduct yourself appropriately in all teaching sessions and to contribute to a positive, inclusive, and productive learning environment. Your behaviour affects not only your own learning, but also the ability of other students and teaching staff to participate effectively.

You should arrive punctually and be ready to participate at the scheduled start time. Arriving late can disrupt the session and distract other students, so you should make every reasonable effort to arrive on time. If you do need to arrive late, you should do so as quietly and unobtrusively as possible.

You are expected to treat your fellow students, teaching staff, technical staff, and other members of the University community with courtesy and respect. You should listen when others are speaking, allow others to contribute, and respect that your classmates may have different backgrounds, experiences, perspectives, and levels of prior knowledge. Behaviour that intimidates, disrupts, excludes, or prevents others from learning is not acceptable.

You are expected to engage actively and constructively with teaching and learning activities. This includes listening and contributing during lectures, participating in discussions and activities during tutorials, and working safely and collaboratively during laboratory sessions. You should ask questions when you need clarification and should be willing to consider and respond to the ideas and contributions of others. In group activities, you are expected to contribute fairly and to support a constructive working environment.

You may use laptops, tablets, phones, and other technology during teaching sessions where this supports your learning or is required for the activity. However, your use of technology should not distract you or others from learning. You should not use devices for activities that are unrelated to the session where doing so is disruptive or prevents you from participating effectively. Teaching staff may provide specific instructions about when and how devices should be used during particular sessions, and you are expected to follow these instructions.

In laboratory sessions, you must follow all instructions relating to health and safety, the use of equipment and computing facilities, and the appropriate use of University systems. You should take reasonable care of the facilities and equipment provided to you and report any problems or damage promptly. You must not undertake activities that could compromise the security, integrity, or availability of University computing systems or interfere with the work of other students.

You should also be mindful of the learning environment when taking breaks, leaving a session, or returning to it. Where you need to leave a session temporarily, do so with as little disruption as possible.

Ultimately, you are responsible for contributing to an environment in which everyone can learn effectively. Professional and respectful conduct is an important part of studying computer science and will help you develop the behaviours and collaborative skills expected in academic and professional environments.

Teaching staff may ask you to adjust your conduct if they consider that it is affecting the learning environment. In more serious or persistent cases, you may be asked to leave the session. You are expected to follow such instructions promptly and respectfully. Any concerns about the instruction should be raised with the appropriate member of staff after the session,

rather than disrupting teaching.

Monitoring engagement

The University actively monitors your attendance and engagement. Your attendance at timetabled activities is recorded through the University's attendance monitoring system. Further information about attendance monitoring, including how attendance is recorded and what you should do if you are unable to attend, is available on the University's website:

<https://students.leeds.ac.uk/attendance-absences>

Attendance monitoring is intended to support you as well as to identify patterns of non-attendance. If you are regularly absent or your engagement gives cause for concern, the University or programme team may contact you to discuss your circumstances and identify whether you need any additional support.

You should therefore not regard attendance monitoring simply as an administrative requirement. It provides an opportunity for the University to identify when you may be experiencing academic, personal, health, financial, or other difficulties and to help you access appropriate sources of support.

Consequences of poor attendance

Regular non-attendance can have a significant impact on your academic progress and your wider university experience. If you do not attend teaching activities, you may miss essential explanations, practical work, opportunities to ask questions, feedback, and collaborative learning with your peers. Persistent non-attendance can therefore make it more difficult for you to meet the learning outcomes of your modules and succeed in your assessments.

If you have a pattern of poor attendance or engagement, the programme team may contact you to discuss your circumstances and your plans for re-engaging with your studies. Where appropriate, you may be referred to relevant University support services or asked to take specific steps to improve your attendance and engagement.

You should be aware that persistent non-attendance may also have consequences under University regulations and procedures, including where attendance or engagement is a requirement for particular modules, placements, professional activities, or continued registration on your programme. If you are unable to attend regularly because of circumstances affecting your studies, you should seek advice at the earliest opportunity rather than allowing non-attendance to continue without explanation.

The purpose of responding to poor attendance is not simply punitive. Early intervention allows the University to understand the reasons for your non-attendance, provide appropriate support, and help you re-engage with your studies wherever possible. If you are experiencing difficulties that are affecting your attendance or engagement, you are encouraged to communicate with your academic or personal tutor and make use of the University's support services.

4 Indicative Course Content

The term "Develop" should be taken to mean "design, write and test".

1 Programming principles

Candidates should be able to:

Indicative content/Notes

- 1.1 Compare and contrast different programming paradigms and explain their appropriate use in different problem contexts.
- 1.2 Explain the differences between compiled and interpreted programming languages, including their respective benefits and limitations.
- 1.3 Explain and compare different execution models used by programming languages.
- 1.4 Apply appropriate programming constructs to represent and transform data using different programming languages.
- 1.5 Explain the role and purpose of data structures in the development of computer programs.
- 1.6 Select and justify appropriate data structures for a given programming problem.
- 1.7 Develop programs that effectively use structured data types provided by a programming language.
- 1.8 Develop programs that follow appropriate coding conventions, including layout, naming, and stylistic practices.
- 1.9 Document programs appropriately using comments and relevant language features.
- 1.10 Analyse a given program and explain its purpose, structure, and behaviour.
- 1.11 Use appropriate programming terminology to identify and describe the elements and constructs within a program.
- 1.12 Trace and analyse the flow of control during the execution of a program.
- 1.13 Explain and distinguish between static, dynamic, and performance testing approaches and select appropriate testing approaches for a given programming context.

2 Python programming

Candidates should be able to:

Indicative content/Notes

- 2.1 Develop Python programs using fundamental programming constructs, including variables, assignment, expressions, input and output, conditional statements, and iterative statements.
- 2.2 Develop Python programs using functions, including parameters, return values, local and global scope, and appropriate decomposition of problems into reusable functions.
- 2.3 Develop Python programs using built-in data structures, including lists, tuples, sets, and dictionaries, and select appropriate data structures for a given problem.
- 2.4 Manipulate textual data using Python strings and appropriate string-processing operations and methods.
- 2.5 Explain the concept of recursion and develop recursive Python functions for appropriate problem types.
- 2.6 Develop Python programs that read from and write to text files to provide persistent data storage.
- 2.7 Use Python modules and standard-library facilities to extend program functionality and select appropriate library functionality for a given problem.
- 2.8 Develop Python programs that use exceptions to detect and handle appropriate runtime errors.
- 2.9 Develop Python programs that follow appropriate Python coding conventions, including meaningful naming, indentation, documentation, and modular program structure.

3 C programming

Candidates should be able to:

Indicative content/Notes

- 3.1 Develop C programs using fundamental programming constructs, including variables, data types, assignment, expressions, input and output, conditional statements, and iterative statements.
- 3.2 Develop C programs using functions, including parameters, return values, local variables, and appropriate decomposition of problems into functions.
- 3.3 Use C arrays to store and manipulate collections of data and select appropriate array structures for a given problem.
- 3.4 Manipulate textual data using C character arrays and appropriate string-handling functions.
- 3.5 Explain the relationship between pointers, addresses, and variables in C, and develop programs that use pointers appropriately.
- 3.6 Use pointers to pass data to functions and to manipulate arrays and other data structures.
- 3.7 Explain the purpose of dynamic memory allocation and develop C programs that dynamically allocate and release memory using appropriate memory-management functions.
- 3.8 Define and use C structures to represent related data and develop programs that manipulate structured data.
- 3.9 Explain the concept of recursion and develop recursive C functions for appropriate problem types.
- 3.10 Develop C programs that read from and write to files to provide persistent data storage.
- 3.11 Use appropriate C libraries and header files to extend program functionality.
- 3.12 Develop C programs that identify and handle appropriate runtime errors and conditions that may result in undefined or erroneous behaviour.
- 3.13 Compile, test, and debug C programs using appropriate compiler options and diagnostic information.

- 3.14 Develop C programs that follow appropriate coding conventions, including meaningful naming, indentation, comments, and modular program structure.

4 Database & Structured Query Language (SQL)

Candidates should be able to:

Indicative content/Notes

- 4.1 Describe the five stages of the data life cycle and explain their relevance to the management of data.
- 4.2 Identify and explain the principal components of a database system, including databases, database management systems, users, and applications.
- 4.3 Explain the key components and concepts of the relational data model, including relations, attributes, tuples, domains, and keys.
- 4.4 Construct a conceptual data model to represent the entities, attributes, and relationships within a specified problem domain.
- 4.5 Derive a logical data model from a conceptual data model, applying appropriate rules for transforming entities and relationships into a relational structure.
- 4.6 Derive relations and appropriate primary and foreign keys from a logical data model.
- 4.7 Identify and distinguish between the common normal forms and apply normalisation principles to reduce data redundancy and potential update anomalies.
- 4.8 Construct SQL queries to retrieve, filter, sort, and combine data from one or more database tables.

5 Assessment & Feedback

Assessments structure

There are two types of assessment used in this module, **formative** and **summative** assessment. Formative assessment is a type of evaluation used in education to monitor student learning and provide ongoing feedback that can be used by instructors and students to improve teaching and learning. Formative assessments will be used by the lab and tutorial leaders to assess your understanding and to guide your learning. Summative assessment is a type of evaluation used to measure student learning, knowledge, proficiency, or achievement at the end of an instructional period. This type of assessment is designed to determine if the learning outcomes and standards of the curriculum have been met. Summative assessment will be used through the module via the portfolio and at the end of the teaching period by terminal assessment.

This module assessment has two components, you will be given an opportunity to attempt both components. The two components are a **portfolio** and **terminal assessments**. You are required to submit and pass the portfolio in order to be awarded credit for this module.

Assessment	Notes	Weight	Timeline
Portfolio	Continuous assessment	N/A	See Table 6 & Table 7
Terminal Assessment - Mid module - End of module	Closed book examination	100% 33.3% 66.7%	January May/June

Portfolio

The portfolio is a collection of exercises which are submitted via Minerva throughout the academic year. Each week you will receive a worksheet which will clearly identify which exercises are part of the portfolio. The portfolio exercises are designed to assess the learning outcomes at a threshold level, that is, by completing the portfolio exercises you will demonstrate your knowledge, skills and behaviours at the **minimum** standard required to pass. To fully prepare for the terminal assessment you should complete all portfolio exercises and follow all additional guidance provided by a member of the teaching staff.

Each set of portfolio exercises are graded as Pass or Fail. You must pass all portfolio exercises to pass the portfolio.

Each worksheet has a due date. You should complete the worksheet and submit the work before the indicated due date. Late submissions are allowed for portfolio exercise up to the late submission date. A portfolio exercise is considered to be on time if it is submitted to the specified submission portal between the release date/time and the due date/time, otherwise it is considered to be a late submission. You **must** submit 6 out of the 9 semester 1 portfolio exercises on time, and 5 out of the 8 semester 2 portfolio exercises on time. If you exceed the limit of late submissions then you will fail the portfolio and you will be required to resit the portfolio.

If you do not submit portfolio exercises then you will be followed up by the School of Computer Science Academic and Pastoral support teams.

Portfolio Identifier	Week	Release date	Due date	Late submission date
S1.1	1	0900 28/09/2026	1400 09/10/2026	1400 22/01/2027
S1.2	2	0900 05/10/2026	1400 16/10/2026	1400 22/01/2027
S1.3	3	0900 12/10/2026	1400 23/10/2026	1400 22/01/2027
S1.4	4	0900 19/10/2026	1400 30/10/2026	1400 22/01/2027
S1.5	5	0900 26/10/2026	1400 06/11/2026	1400 22/01/2027
Reflection week	6			
S1.6	7	0900 09/11/2026	1400 20/11/2026	1400 22/01/2027
S1.7	8	0900 16/11/2026	1400 27/11/2026	1400 22/01/2027
S1.8	9	0900 23/11/2026	1400 04/12/2026	1400 22/01/2027
S1.9	10	0900 30/11/2026	1400 11/12/2026	1400 22/01/2027

Table 6: Semester 1 portfolio dates.

Portfolio Identifier	Week	Release date	Due date	Late submission date
S2.1	1	0900 25/01/2027	1400 05/02/2027	1400 07/05/2027
S2.2	2	0900 01/02/2027	1400 12/02/2027	1400 07/05/2027
S2.3	3	0900 08/02/2027	1400 19/02/2027	1400 07/05/2027
S2.4	4	0900 15/02/2027	1400 26/02/2027	1400 07/05/2027
S2.5	5	0900 22/02/2027	1400 05/03/2027	1400 07/05/2027
S2.6	6	0900 01/03/2027	1400 12/03/2027	1400 07/05/2027
S2.7	7	0900 08/03/2027	1400 19/03/2027	1400 07/05/2027
S2.8	8	0900 15/03/2027	1400 26/03/2027	1400 07/05/2027

Table 7: Semester 2 portfolio dates.

Terminal assessment

The terminal assessment is an assessment which takes place at the end of a teaching period.

Feedback approach

Feedback refers to the information provided to you about your performance on various academic tasks, such as assignments, exams, projects, or class participation. The purpose of feedback is to help you understand your strengths and areas for improvement, guiding your learning journey and enhancing your academic skills.

You will receive feedback on the formative activities on the weekly worksheets. Feedback will be provided in the tutorial or lab classes. During the lab sessions and tutorials you will be able to engage in dialogue with a teaching assistant to receive feedback. Some feedback may be written, depending on the nature of the activities.

You will be provided with an indication of performance on summative portfolio exercises. You can request feedback on your terminal assessment by contacting the module leader. You will not be able to request a copy of your marked terminal assessment.

Understanding your grade

Your grade is made up of two components and will be available in student service (Minerva) after the release date. The two components are the **numeric component** and the **suffix component**.



The numeric component of your grade indicates the value that will be incorporated into your classification average calculation, where appropriate (See [Academic Regulations](#)).

Numeric component

Table 8 summarises the possible numeric component values.

Numeric component	meaning
0-39	You have failed the module. You have not been awarded credits.
40-100	Any grade between 40-100 is a pass grade (see suffix component table below to determine if credits have been awarded).

Table 8: Values of the numeric component.

Suffix component

Table 9 summarises the possible suffix component values.

Suffix	meaning
None	Your grade only contains a numeric component. You have been awarded credits if your numerical grade is 40 or above.
I	You have an outstanding academic integrity investigation concerning the work that you have submitted. You have not been awarded credits.
H	You have an unexpunged academic integrity penalty. You are required to expunge the penalty, please refer to your outcome letter. You have not been awarded credits.
G	You have satisfactorily expunged an academic integrity penalty and you have been awarded credit for the module.

Table 9: Values of suffix component.

You must pass the portfolio to be awarded credit for this module. Your module grade will be calculated from your terminal assessment mark. If you pass the portfolio component then your module grade will be your terminal assessment mark. If you fail the portfolio component then your module grade will be your terminal assessment mark capped at 39.

Assessment timeline & Late submissions

The examination timetable will be published around Easter for the May/June examination period. At this point the date, time and location of your examinations will be available to you. Your examinations will appear in your personalised timetable. If you have reasonable adjustments due to a disability support summary sheet then you may be in a different room to the main cohort.

Resit opportunity

At the University of Leeds you are ordinarily granted two attempts to pass a module. Your first attempt will allow you to achieve a grade between 0 and 100. Your second attempt is capped, allowing you to achieve a maximum grade of 40. If you have passed a module then you are not permitted to resit it.

Under some circumstances you may be eligible for a further attempt, or a first attempt resit. Eligibility is subject to an approved mitigating circumstances application. Information about mitigating circumstances can be found in the [Mitigating circumstance guidance](#).

If you have not been awarded credits for the module then you will have an opportunity to resit the module. You must take this resit opportunity at the next available point in time that the assessment is offered. The next available opportunity is during the August resit period. You must register to take the resit. Information about how to register will be provided by email after you have received your results.

To resit the module you will be required to complete the portfolio to a satisfactory standard. The resit attempt will be **capped** at the pass grade 40. If you have applied for, and have been granted a first attempt resit then you will also have an opportunity to resit the terminal assessment for an uncapped grade.

Note you must submit and pass the portfolio in order to be awarded credit for this module.

6 Academic Integrity

Academic integrity statement

Academic integrity is a commitment to good study practices and shared values, which ensures that your work is a true expression of your own understanding and ideas, giving credit to others where their work contributes to yours.

This University definition of academic integrity recognises that each individual has a responsibility to contribute honestly within our academic community. Breaching academic integrity standards can lead to serious penalties. Make sure that you are familiar with the information in the pages below, [complete the relevant Academic Integrity Tutorials and Tests](#), and develop your understanding of good study practices through the [Skills@Library](#) webpages to ensure that your own work is your best work. Learn more about our definition of academic integrity by exploring the headers and links below.

Declaration of academic integrity

When you submit for assessment, you are declaring to the University that your work is your own. By submitting work, you confirm that you have read and understood the University's definition of Academic Integrity and the University's rules on Academic Misconduct and are aware that if you commit an academic misconduct offence there can be serious disciplinary consequences.

For more information about the University expectations of working with academic integrity and support available to you see the [Library's academic integrity webpage](#). If you have concerns, or would like support or guidance, please contact your personal tutor and/or your module tutor for advice.

For more information on academic misconduct offences, see the [Academic Misconduct Procedure](#).

Use of assistive technologies

Unless otherwise stated you may use assistive technologies such as grammar checkers, spelling correctors, generative AI tools to help you study and learn but you are not permitted to use the tools in the creation of work submitted for assessment.

All learning resources and assessment materials are the property of the University of Leeds. You are not permitted to submit materials to Artificial Intelligence systems.

7 Reading & Additional resources

Reading list

Sections from the books below are suggested for certain weeks of teaching. Programming books come in many forms and styles. You may prefer an alternative and that is fine, as the content is usually quite similar. For example, the Head First books take quite a different approach that some people prefer.

All books are available from the University Library, in print and online. See the Minerva module Reading List for direct links.

“How to think like a computer scientist : learning with Python”,
Downey, Allen, Elkner, Jeff; Meyers, Chris,
Green Tea Press, 2008.

“Head First Python”,
Barry, Paul,
Second edition., O’Reilly Media, 2017.

“Database systems : a practical approach to design, implementation and management”,
Connolly, Thomas M., Begg, Carolyn,
Sixth edition, Pearson Education Limited, 2015.

“Programming in C””,
Kochan, Stephen G.,
Fourth edition., Addison-Wesley, 2015.

Additional resources

Programming platforms and tools: Linux, Git, Github, Codepsaces and VScode, are introduced as part of the module. We will usually point to standard documentation for these, as part of your general development.

8 Accessibility

Accessibility statement

We have taken great care to avoid bias of any kind in the preparation of this module and related assessment materials. In compliance with the UK Equality Act (2010) we have designed this module to avoid any direct and indirect discrimination.

The standard assessment arrangements may present unnecessary barriers for candidates with disabilities or learning difficulties. Reasonable adjustments are available to students to enable them to access the assessments and receive recognition of their attainment. Reasonable adjustments will only be agreed where they do not give students an unfair advantage over others or compromise the standards being assessed.

Students who cannot access the assessment of any component may be able to receive an award based on the parts of the assessment they have completed

Requesting reasonable adjustments to learning resources

You can request reasonable adjustments to learning resources by receiving a support summary sheet from the University of Leeds Disability Services. Your support summary sheet will be shared with the module leaders and other relevant teaching staff. You may wish to speak individually to the module leader to discuss how the adjustments will be implemented from your support summary sheet.

Requesting reasonable adjustments in assessments

You can request reasonable adjustments to assessments by receiving a support summary sheet from the University of Leeds Disability Services. The University of Leeds will automatically implement any adjustments indicated on your support summary sheet for examination based assessments. Your support summary sheet will be shared with the module leader and other relevant teaching staff. You may wish to speak individually to the module leader to discuss how the adjustments will be implemented from your support summary sheet concerning assessments.